

UNIVERSIDAD POLITÉCNICA DE MADRID · ETSIINF

Aplicación multiplataforma y PWA para la gestión integral de un campus de baloncesto

Trabajo de Fin de Grado · Grado en Ingeniería de Software

Autor: Alberto Rodríguez González **Tutor:** José Ramón Sánchez Couso

Madrid, junio de 2026

NOTAS · DIAPOSITIVA 1

Buenos días. Soy Alberto Rodríguez González y presento mi Trabajo de Fin de Grado, tutorizado por José Ramón Sánchez Couso: Campus Baloncesto App, una aplicación multiplataforma y PWA para gestionar de forma integral un campus de baloncesto. Surge de mi experiencia como entrenador en el campus que se celebra en Toledo cada julio, donde cerca de cien chavales de distintas edades y niveles entrenan, compiten y hacen vida de campamento durante once días. En los próximos minutos os explicaré el problema que resuelve, las decisiones técnicas, la arquitectura, las funcionalidades —con foco en el tiempo real— y los resultados. La aplicación está además desplegada y en uso, accesible en campus-baloncesto.web.app.

En la memoria:

resumen y estructura general del documento (apartado 1.4).

Índice

01

Contexto y motivación



02

Objetivos y
planificación



03

Estado del arte y
tecnologías



04

Arquitectura y
funcionalidades



05

Tiempo real y
exportaciones



06

Pruebas y resultados



NOTAS · DIAPOSITIVA 2

Seguiré este guion, que coincide con la estructura de la memoria: primero el contexto y los objetivos; después el estado del arte y las decisiones tecnológicas con su arquitectura; luego las funcionalidades principales, deteniéndome en la generación de equipos, el tiempo real y las exportaciones; y cerraré con las pruebas, los resultados y las conclusiones. La idea es dar una visión completa del proyecto sin entrar en cada detalle, que queda recogido en el documento.

En la memoria:

el documento sigue esta misma estructura por capítulos (apartado 1.4, «Estructura del documento»).

Contexto y motivación

Soy entrenador del campus de baloncesto de verano de Toledo.

Cada año la gestión se hacía con hojas de cálculo, grupos de WhatsApp y papel: información dispersa, lenta de actualizar y difícil de compartir con familias y jugadores.

«Una sola herramienta, accesible desde el móvil, la tablet o el navegador, sin instalar nada distinto en cada dispositivo.»



Hojas de cálculo

Clasificaciones manuales y propensas a errores



WhatsApp

Avisos y resultados perdidos entre mensajes



Papel

Puntuaciones y equipos apuntados a mano

NOTAS · DIAPOSITIVA 3

El campus reúne cada edición a cerca de un centenar de jugadores de distintas edades y niveles, y toda su gestión se apoyaba en herramientas que no se hablan entre sí: una hoja de cálculo para las clasificaciones, un grupo de WhatsApp por equipo, folios para las estadísticas y una pizarra para los resultados. Eso genera errores, duplica trabajo y da una imagen poco profesional; la información se queda vieja enseguida y la comunicación se vuelve un caos con treinta o cuarenta personas en un grupo. A esto se sumó que la propia organización quería dar un salto informático y modernizar su imagen. La solución fue centralizarlo todo en una sola aplicación accesible desde cualquier dispositivo: los administradores trabajan desde el ordenador y entrenadores, jugadores y familias desde el móvil. Por 'gestión integral' entiendo cubrir el ciclo concreto del campus de principio a fin; los pagos y la facturación los dejé conscientemente fuera, como línea futura.

En la memoria:

capítulo 1.1 «Contexto y motivación»; la planificación temporal está en el apartado 1.5 y la figura 1.1 (diagrama de Gantt).

Objetivos del proyecto

Objetivo general Digitalizar la gestión integral del campus en una única aplicación multiplataforma (Android, iOS y web/PWA).



Gestionar usuarios y equipos

Roles, perfiles y grupos del campus



Competiciones y rankings

Estaciones, siesta, veladas y partidos



Resultados en tiempo real

Marcador en directo y estadísticas



Seguridad por roles

Acceso controlado con políticas RLS

NOTAS · DIAPOSITIVA 4

El objetivo general era diseñar y desarrollar esa aplicación multiplataforma —app nativa en Android e iOS y PWA en navegador— que sustituyera lo manual por algo unificado, seguro y en tiempo real. De ahí salen los objetivos específicos: un sistema de autenticación con seis roles; la gestión de equipos con generación automática equilibrada; las competiciones con fase de grupos, eliminatorias y veladas, más el seguimiento en directo; el registro de estadísticas —puntos, rebotes, asistencias, robos y tapones— con sus rankings y el MVP; la gamificación y el jugador premium; los canales de comunicación; las dos exportaciones; la sincronización en tiempo real; las notificaciones push; y el despliegue como PWA instalable de acceso público.

En la memoria:

capítulo 1.2 «Objetivos» y 1.3 «Alcance».

Metodología y planificación



Iterativa e incremental

Módulo a módulo, probando cada parte al terminarla.



Git + GitHub

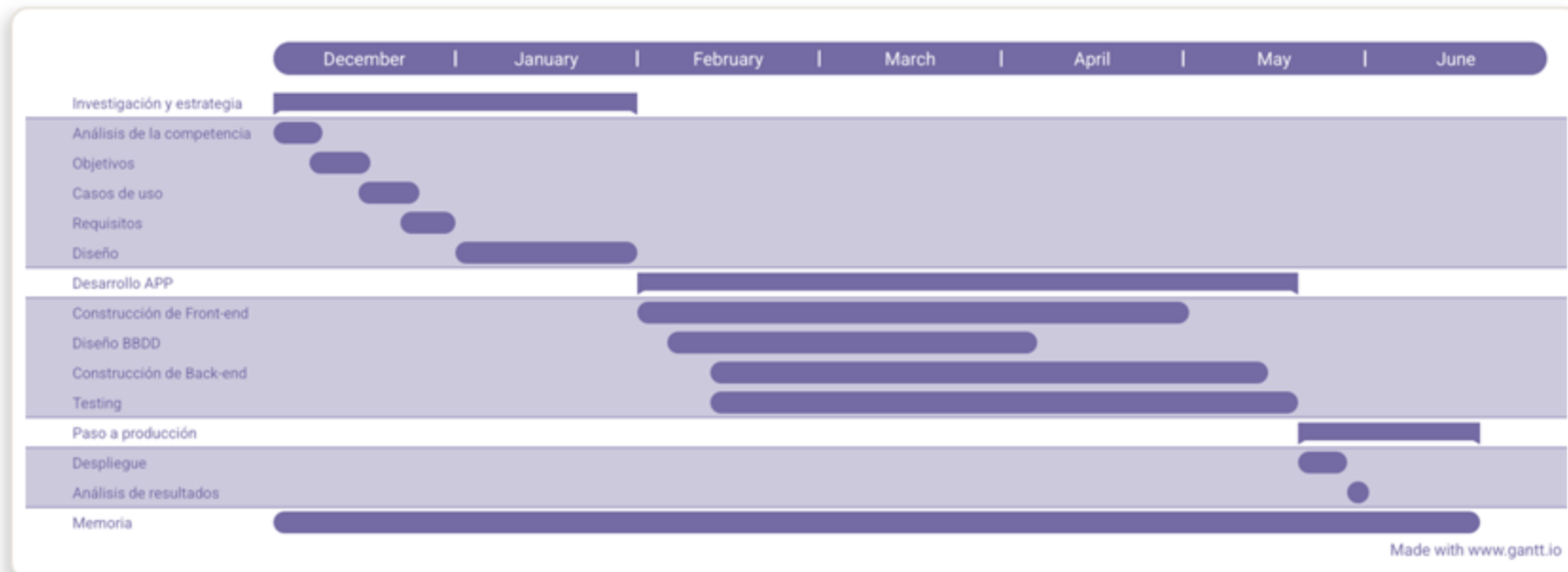
Control de versiones y repositorio público.



Dic 2025 - Jun 2026

Análisis, diseño, desarrollo, pruebas y memoria.

PLANIFICACIÓN TEMPORAL · DIAGRAMA DE GANTT



NOTAS · DIAPOSITIVA 5

Trabajé con una metodología iterativa e incremental: en lugar de un único bloque, fui construyendo la aplicación módulo a módulo y probando cada funcionalidad nada más terminarla, ajustando sobre la marcha. El desarrollo se planificó de diciembre de 2025 a junio de 2026, y el diagrama de Gantt resume las fases —investigación y análisis, diseño, desarrollo por capas (front-end, base de datos y back-end), pruebas, paso a producción y memoria—, que en parte se solaparon. Como entorno usé Flutter con VS Code y control de versiones con Git, con el código alojado en un repositorio público de GitHub. Este enfoque, con entregas pequeñas y verificación continua, encajaba muy bien en un proyecto llevado por una sola persona.

En la memoria:

apartado 1.5 «Planificación del proyecto» (figura 1.1, diagrama de Gantt) y 5.2 «Metodología y entorno de desarrollo».

Estado del arte

Las apps deportivas comerciales (TeamSnap, GameChanger...) son potentes, pero genéricas, de pago y no cubren las competiciones por estaciones, las veladas ni la siesta. Comparé también los frameworks multiplataforma y los servicios backend disponibles:

FRAMEWORKS

Framework	Punto clave
Flutter ✓	Compila a nativo · 1 base de código
React Native	Puente JS · menos fluido
Ionic	WebView · rendimiento bajo

BACKEND (BaaS)

Backend	Punto clave
Supabase ✓	PostgreSQL · RLS · tiempo real
Firebase	NoSQL · complica lo relacional
AWS Amplify	Potente · configuración compleja

De ahí salen las dos decisiones clave del proyecto: Flutter y Supabase.

NOTAS · DIAPOSITIVA 6

Antes de decidir nada miré qué había. Por un lado, aplicaciones de gestión deportiva ya existentes, del estilo de TeamSnap o GameChanger: son potentes, pero están pensadas para ligas genéricas, suelen ser de pago y no encajan con las particularidades de un campus, que tiene competiciones por estaciones, veladas y competiciones de siesta. Por otro lado, comparé los frameworks multiplataforma —Flutter, React Native e Ionic— y los servicios backend —Supabase, Firebase y AWS Amplify—, valorando rendimiento, idoneidad del modelo de datos, coste y mantenibilidad. De ese análisis salieron las dos decisiones clave que veréis a continuación: Flutter para el cliente y Supabase para el backend.

En la memoria:

capítulo 2 «Estado del Arte»: aplicaciones de gestión deportiva (2.1–2.2, tabla 2.1), frameworks (2.3, tabla 2.2) y servicios backend (2.4).

¿Por qué este stack?



Flutter

Una sola base de código compila a Android, iOS y web. Frente a React Native o desarrollo nativo, máxima reutilización.



Supabase

Backend gestionado (PostgreSQL, Auth, Realtime y RLS). Más control relacional que Firebase para los datos del campus.



PWA

La versión web se instala como app y funciona sin tiendas, clave para que cualquiera la use al instante.

NOTAS · DIAPOSITIVA 7

Comparé las opciones en vez de elegir lo que más me sonaba. Para el cliente me decanté por Flutter, que era el que mejor equilibraba rendimiento nativo, consistencia visual entre plataformas y un soporte web suficientemente maduro, todo desde una sola base de código; frente a React Native o a Ionic, que corre sobre un WebView y pierde fluidez. Para el backend elegí Supabase, construido sobre PostgreSQL, porque me daba una base de datos relacional completa, seguridad a nivel de fila, tiempo real y autenticación ya hechos; para un dominio tan relacional como un campus encajaba mucho mejor que el NoSQL de Firebase. Acabé en una estrategia mixta: Supabase como backend principal, Firebase Cloud Messaging para las notificaciones, Firebase Hosting para la PWA y Cloudinary para las imágenes. Además, al ser Supabase de código abierto y con planes gratuitos generosos, abarata el mantenimiento y evita el vendor lock-in, algo crítico para una organización de base.

En la memoria:

apartado 2.6 «Justificación de la solución propuesta», apoyado en las comparativas del capítulo 2 (tablas 2.1 y 2.2).

Arquitectura del sistema

Clean Architecture por funcionalidades (feature-first), con la regla de dependencia hacia el dominio:



STACK TÉCNICO

-  Flutter 3 / Dart
-  Riverpod · go_router
-  Supabase (PostgreSQL)
-  Firebase Cloud Messaging
-  Cloudinary (imágenes)
-  Firebase Hosting (PWA)

NOTAS · DIAPOSITIVA 8

La aplicación sigue una Clean Architecture pero organizada por funcionalidades: en lugar de carpetas globales por tipo, cada módulo guarda sus propias capas de presentación, dominio y datos, con la dependencia siempre hacia el dominio. El código se divide en lib/core, con lo compartido —configuración, tema, enrutador y servicios globales—, y lib/features, con una carpeta por funcionalidad. El estado se gestiona con Riverpod y sus AsyncNotifier, cuyo tipo AsyncValue representa de forma explícita los estados de carga, dato y error; además los proveedores se componen entre sí, de modo que, por ejemplo, la clasificación se recalcula sola cuando cambian los partidos. La navegación es declarativa con go_router, que en la PWA refleja las rutas en la URL y permite enlaces directos, con guardas de acceso según la sesión. El stack completo está a la derecha.

En la memoria:

capítulo 4: Clean Architecture (apartado 4.1, figura 4.1 y tabla 4.1), Riverpod (4.2, figura 4.2) y GoRouter (4.3, figura 4.3); el stack en la tabla 5.1 y el despliegue en la figura 4.5.

Modelo de datos y seguridad

20
tablas en PostgreSQL
Modelo relacional normalizado:
usuarios, equipos, competiciones,
partidos, veladas, estadísticas...

**Seis roles**
Administrador, entrenador, jugador, jugador premium, familiar y visitante.

**Row Level Security**
Políticas en la base de datos: cada rol solo ve y edita lo que le corresponde.

**Autenticación Supabase**
Registro e inicio de sesión gestionados con tokens y trigger de alta automática.

NOTAS · DIAPOSITIVA 9

El modelo son 20 tablas en PostgreSQL gestionadas por Supabase, con un diseño marcadamente relacional: tablas de entidad y de relación, como `team_members`, que modela la pertenencia de jugadores a equipos en muchos-a-muchos. La seguridad se apoya en dos pilares: la autenticación de Supabase y, sobre todo, las políticas de Row Level Security definidas en la propia base de datos. Cuando alguien se registra, un trigger crea su fila con el rol 'visitante' y un administrador lo promociona después. Defino seis roles —administrador, entrenador, jugador, jugador premium, familiar y visitante—, cada uno con un conjunto coherente de permisos. La clave de la RLS es que la autorización vive en el servidor: cada política es una expresión booleana que el motor evalúa fila a fila, así que la seguridad se mantiene aunque alguien acceda directamente a la API.

En la memoria:

apartado 4.5 «Modelo de datos» (figura 4.4 y tabla 4.2), 4.6 «Diseño de seguridad y roles» y el detalle de tablas en el Anexo A (tabla A.1).

Funcionalidades principales



NOTAS · DIAPOSITIVA 10

Estas son las funcionalidades principales, derivadas de los casos de uso y de los requisitos: gestión de equipos y usuarios, generación automática de equipos, los formatos de competición, los partidos en directo, las estadísticas y rankings, el dashboard de administración, las exportaciones a PDF y Excel y las notificaciones push. Cada una está implementada como un módulo independiente dentro de lib/features. A continuación me detengo en las más representativas, que son también las que más aportan frente a las herramientas que se usaban antes.

En la memoria:

capítulo 5 «Implementación» en conjunto, y los casos de uso del apartado 3.4 (figura 3.1).

Equipos equilibrados con un toque

El reto: repartir a los jugadores en equipos parejos para que los partidos sean competidos.



Algoritmo de equilibrado

Reparte según nivel, edad y posición para igualar la fuerza de cada equipo.



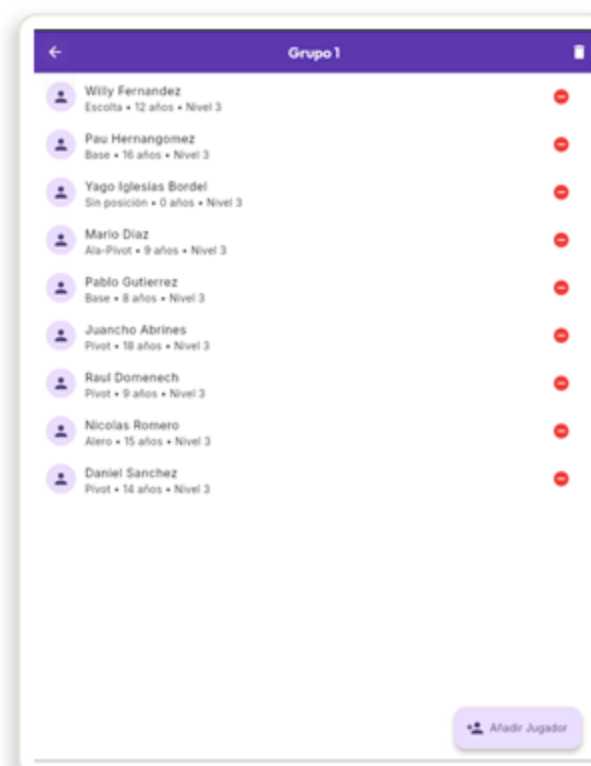
Equipos de partido

Generados aparte y excluidos del ranking de competición (is_match_team).



Manual o automático

El entrenador puede crear grupos a mano o generarlos al instante.



NOTAS · DIAPOSITIVA 11

Uno de los problemas clásicos del campus era repartir a los jugadores en equipos parejos para que los partidos fueran competidos. El generador automático, implementado en el método `autoGenerateBalancedTeams`, parte de la edad, el nivel y la posición de cada jugador. Asigna a cada uno una puntuación de fuerza con una fórmula sencilla —el nivel pesa diez veces más que la edad: $\text{nivel por diez, más la edad}$ —, separa a los jugadores por posiciones (pívots, bases y aleros) y reparte cada bloque, ordenado de más fuerte a más débil, con una heurística voraz tipo serpiente. En el fondo es un problema de partición equilibrada: una asignación al azar daría equipos descompensados y una optimización exhaustiva sería carísima, así que esta heurística logra un buen resultado a coste muy bajo. Además existen los equipos de partido, que se generan aparte y se excluyen del ranking de competición mediante el flag `is_match_team`.

En la memoria:

apartados 5.4 «Gestión de equipos» y 5.5 «Generación automática» (caso de uso CU-01; figuras 5.5 a 5.7).

Tres formatos de competición



Por estaciones

Pruebas físicas y técnicas con puntuación por jornadas y clasificación general.



Competiciones de siesta

Ligas y eliminatorias (grupos → cuadro) con clasificación y brackets.



Veladas

Grupos por equipos con ganadores, pensado para las actividades de tarde-noche.

Clasificación / Eliminatorias					
Clasificación			Eliminatorias / Cuadro		
GRUPO 2			GRUPOS		
Alvaro Gomez	7		Pau Hernangomez	7	Pablo Gutierrez
Diego Ruiz	5		Raul Domenech	2	Adrian Martin
Hugo Perez	7		Alex Almansa	7	Pau Hernangomez
Diego Ruiz	3		Raul Domenech	3	Hugo Perez
Hugo Perez	7		Adrian Martin	7	
Alvaro Gomez	2		Raul Domenech	2	
Pablo Gutierrez	7		Adrian Martin	7	
Diego Ruiz	0		Alex Almansa	5	
Pablo Gutierrez	7		Pau Hernangomez	7	
Alvaro Gomez	3		Alex Almansa	4	
Pablo Gutierrez	8		Pau Hernangomez	7	
Hugo Perez	2		Adrian Martin	6	

NOTAS · DIAPOSITIVA 12

Hay tres formatos de competición. Las competiciones por estaciones, pruebas físicas y técnicas puntuadas por jornadas con una clasificación general. Las competiciones de siesta, que combinan liga y eliminatorias con su cuadro; aquí tuve que cuidar el desempate, replicando un esquema parecido al de la FIBA cuando dos participantes terminan con los mismos puntos. Y las veladas, actividades por equipos pensadas para la tarde-noche. Son justo el tipo de competiciones singulares de un campus que las herramientas genéricas no cubren bien, y por eso tenía sentido una solución propia.

En la memoria:

apartados 5.6 «Competiciones por estaciones» (figuras 5.8–5.11), 5.7 «Competiciones de siesta» (figuras 5.12–5.15) y 5.8 «Veladas» (figuras 5.16–5.17).

Partidos en directo, en tiempo real



Supabase Realtime

Vía WebSockets, el marcador se sincroniza al instante en todos los dispositivos.



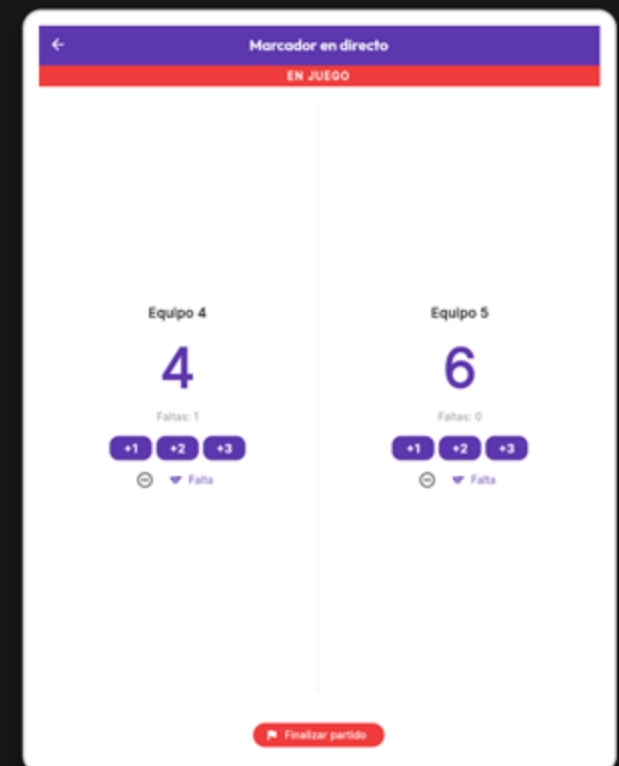
Marcador y faltas

Puntos y faltas por equipo; los partidos en juego se ven desde cualquier pantalla.



Latencia < 1 s

Las actualizaciones llegan a los clientes suscritos en menos de un segundo.



NOTAS · DIAPOSITIVA 13

Esta fue la parte técnicamente más exigente, así que me detengo un poco. Lo normal en la web es el modelo petición–respuesta: el cliente pregunta y el servidor contesta; para enterarte de los cambios tendrías que estar preguntando sin parar. Un WebSocket, en cambio, es una conexión permanente y bidireccional: una vez abierta, el servidor puede empujar los datos al instante sin que el cliente pregunte. Los partidos en directo se apoyan en Supabase Realtime, que usa WebSockets por debajo: cada dispositivo se suscribe a la tabla `live_matches` y, cuando un entrenador suma puntos o faltas, se escribe en esa tabla, PostgreSQL detecta el cambio en su registro de replicación y Supabase lo retransmite por el WebSocket a todos los suscritos. Esas suscripciones se integran con los `StreamProvider` de Riverpod, de modo que el marcador se redibuja solo; medí la latencia y llega en menos de un segundo. Un detalle de diseño importante: aunque haya varios partidos a la vez, no abro una conexión por partido, sino una única suscripción a toda la tabla, y cada marcador filtra su propia fila de ese mismo flujo. Así el sistema escala sin conexiones extra y, al ser cada partido una fila independiente, no interfieren entre sí.

En la memoria:

apartado 5.11 (caso de uso CU-02; figuras 5.21 y 5.22) y el requisito RF-13; la latencia se verifica en el capítulo 6.

Estadísticas y exportaciones



Rankings del campus

Máximo anotador, reboteador, asistente y MVP, calculados automáticamente.



Dashboard de admin

Visión global del campus de un vistazo: personas, competiciones y comunidad.



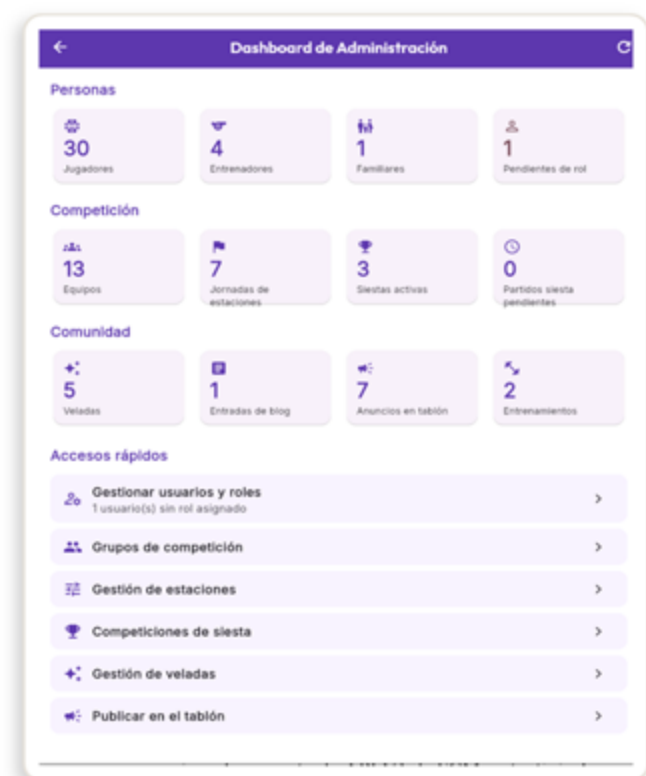
Exportación a PDF

Clasificaciones y cuadros de las competiciones de siesta, listos para imprimir.



Exportación a Excel

Clasificación por estaciones, una hoja por grupo y puntuaciones por jornada.



NOTAS · DIAPOSITIVA 14

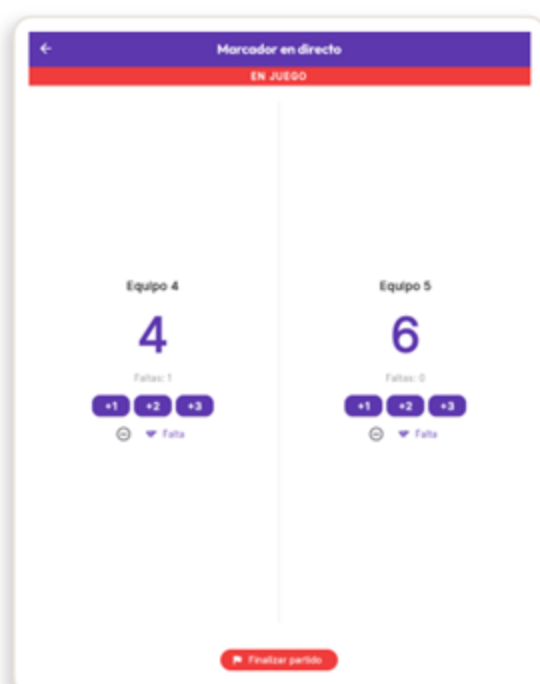
Las estadísticas se registran por jugador y partido en la tabla `player_match_stats` —puntos, rebotes, asistencias, robos, tapones y MVP— y de ahí se calculan los rankings de anotador, reboteador y asistente. El dashboard de administración concentra el control del campus: promocionar a un usuario recién registrado, corregir un equipo o abrir una competición, algo clave durante los días de campus, cuando el ritmo es alto. Y están las dos exportaciones: el servicio `SiestaExportService` genera el PDF con la clasificación y el cuadro de eliminatorias, y `ExportService` produce el Excel de las estaciones con una hoja por grupo y las puntuaciones por día más el total. Su comportamiento cambia entre la web y el móvil, así que las validé por separado en cada plataforma.

En la memoria:

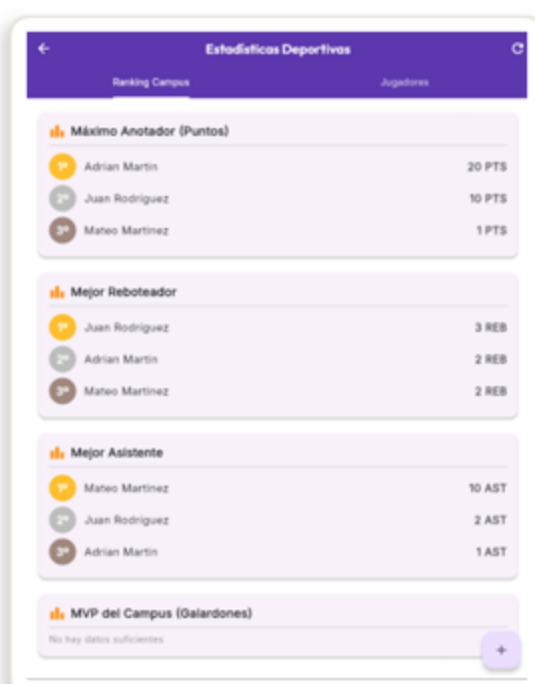
rankings y estadísticas (5.11, figuras 5.23–5.24), dashboard (5.13, figura 5.25) y exportaciones (5.15: PDF en 5.15.1 / figura 5.26, Excel en 5.15.2 / figura 5.27; caso de uso CU-03).

DEMOSTRACIÓN

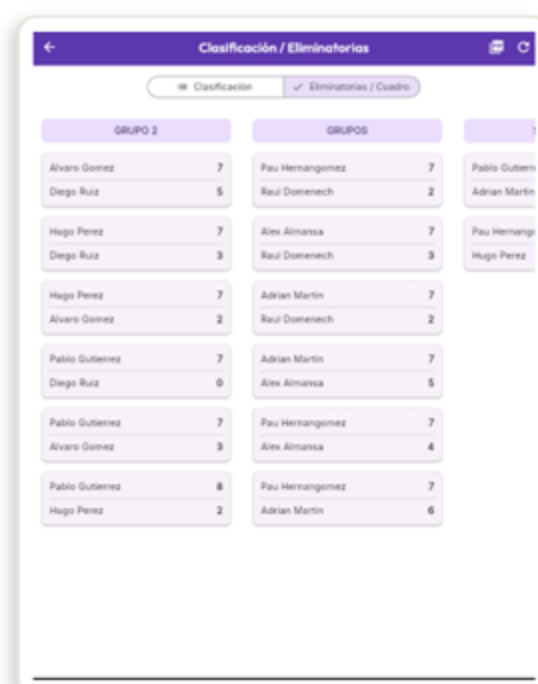
La app, en producción



Marcador en directo



Rankings del campus



Competición de siesta

• EN VIVO campus-baloncesto.web.app

NOTAS · DIAPOSITIVA 15

Aquí se ve la aplicación real: el marcador en directo, los rankings del campus y el cuadro de una competición de siesta. Todas las pantallas están documentadas en la memoria. Está desplegada como PWA en Firebase Hosting y es de acceso público en campus-baloncesto.web.app, instalable desde el navegador sin pasar por las tiendas. (Si el tribunal lo permite, este es el momento de enseñarla funcionando en vivo.)

En la memoria:

las capturas de todas las pantallas están en el capítulo 5 (figuras 5.1–5.27); el despliegue, en el apartado 5.16 y el Anexo B.

Pruebas



Pruebas manuales e incrementales

Al terminar cada funcionalidad se probaba metiendo datos y navegando por las pantallas para validar el comportamiento.



Pruebas beta con entrenadores

Otros entrenadores usaron la app con libertad y destaparon fallos que se me escapaban por conocerla demasiado.



Errores detectados y corregidos

Refrescos que faltaban, límite de correos al registrar, brackets invertidos y un Excel que salía vacío.

NOTAS · DIAPOSITIVA 16

La estrategia de pruebas fue principalmente manual e incremental: al terminar cada funcionalidad la probaba metiendo datos y navegando por las pantallas para validar el comportamiento antes de seguir. Después organicé una fase de pruebas beta con otros entrenadores del campus, que al usar la aplicación con libertad destaparon fallos y detalles de usabilidad que a mí se me escapaban por conocerla demasiado. Gracias a ellos corregí cosas como pantallas que no se refrescaban, un límite de correos al registrar, los cuadros de eliminatorias en orden invertido o un Excel que en algún caso salía vacío. Reconozco que un conjunto de pruebas automatizadas habría dado más garantías, y lo dejo como línea de mejora.

En la memoria:

capítulo 6 «Pruebas»: estrategia (6.2), casos de prueba (tabla 6.1) y pruebas beta con entrenadores (6.5).

Objetivos cumplidos

3plataformas
desde un código**< 2 s**carga de las
pantallas (RNF-02)**< 1 s**actualización
en tiempo real

- ✓ Aplicación desplegada y en uso real (PWA + APK)
- ✓ Gestión completa: usuarios, equipos y competiciones
- ✓ Tiempo real y exportaciones funcionando en las 4 plataformas
- ✓ Todos los requisitos funcionales implementados

NOTAS · DIAPOSITIVA 17

Se cumplieron todos los objetivos planteados: la aplicación gestiona el campus de forma integral, incorpora el algoritmo de generación de equipos, registra estadísticas, sincroniza en tiempo real, envía notificaciones y ofrece las dos exportaciones. En lo técnico, la pareja Flutter más Supabase fue la decisión más acertada: cubre tres plataformas con una sola base de código y delega en el backend la persistencia, la autenticación, el tiempo real y la seguridad. En cuanto al rendimiento, las pantallas cargan en menos de dos segundos —cumpliendo el RNF-02— y las actualizaciones en tiempo real llegan en menos de un segundo. Frente a las alternativas comerciales, el resultado reúne en un único producto, de código abierto y bajo coste, capacidades que esas soluciones solo ofrecen fragmentadas o a precios altos.

En la memoria:

capítulo 7 «Resultados»: estado de los módulos (tabla 7.1) y cumplimiento de objetivos (tabla 7.2); el RNF-02 se verifica en 6.4.

CONCLUSIONES

Un proyecto completo, de la idea al uso real

- ✓ Resuelve un problema real con una sola herramienta multiplataforma
- ✓ Integra tiempo real, seguridad por roles y exportación de datos
- ✓ Demuestra el ciclo completo: análisis, diseño, desarrollo y despliegue

Líneas futuras

Pruebas automatizadas · estadísticas integradas en el propio partido · más analíticas para el entrenador.

¡Gracias! campus-baloncesto.web.app

NOTAS · DIAPOSITIVA 18

Como conclusión, el proyecto demuestra que una sola persona puede construir una aplicación multiplataforma completa y profesional desde una única base de código, recorriendo el ciclo entero de ingeniería: análisis, diseño, desarrollo y despliegue en producción. Las aportaciones de más valor son el algoritmo de equilibrado de equipos y las dos exportaciones, ajustadas a necesidades reales. Por el camino afronté retos como las notificaciones push duplicadas o la adaptación de las exportaciones a cada plataforma. Como líneas futuras quedan las analíticas avanzadas, la inscripción y el pago en línea, una versión para wearables, el soporte multilingüe y, sobre todo, automatizar las pruebas con integración continua. El verdadero examen será la edición de julio de 2026, con el campus a pleno rendimiento. Muchas gracias; quedo a vuestra disposición para las preguntas.

En la memoria:

capítulo 8: conclusiones (8.1) y líneas futuras (8.2).